

Analizador Sintatico (Parser) da Linguagem Java--

Construido com JFlex (analise lexica) e JCup (analise sintatica e semantica)

Disciplina:	Teoria dos Compiladores
Trabalho:	Entrega do Parser (JFlex + JCup)
Ferramentas:	JFlex 1.9.1 - Java CUP 11b - JDK 17
Linguagem-alvo:	Java--

1. Visao Geral

Este trabalho implementa um **Analizador Sintatico (Parser)** para a linguagem **Java--**. O compilador le um arquivo-texto, valida a sua **sintaxe** e executa **acoes semanticas basicas** (Traducao Dirigida pela Sintaxe), como o calculo do resultado de expressoes aritmeticas e a emissao de mensagens de reconhecimento. A construcao segue o fluxo apresentado na Aula 14: o **JFlex** gera o Scanner e o **JCup** gera o Parser; ambos se comunicam pela classe **sym** (os simbolos).

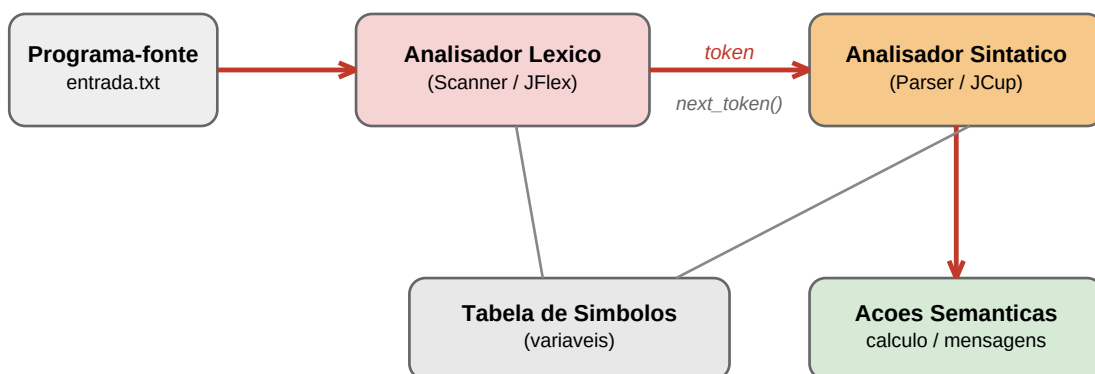


Figura 1 - Fluxo de funcionamento: Scanner (JFlex) -> Parser (JCup) -> Acoes Semanticas.

1.1 Modulos e interdependencias

Modulo	Descricao
Scanner.flex	Especificacao das Expressoes Regulares. E submetido ao JFlex, que gera o Scanner.java.
Scanner.java	Analizador Lexico gerado pelo JFlex. Le os caracteres e devolve tokens (java_cup.runtime.Symbol).
Parser.cup	Especificacao da gramatica + acoes semanticas. Submetido ao JCup.
parser.java	Analizador Sintatico gerado pelo JCup. Verifica a estrutura e dispara as acoes.
sym.java	Gerado pelo JCup. Define as constantes dos terminais (canal de comunicacao Scanner<->Parser).
Main.java	Acopla Scanner e Parser: instancia o Scanner sobre o arquivo e chama p.parse().
entrada.txt	Arquivo-fonte Java-- usado como teste.

2. Tokens e Expressoes Regulares

A tabela abaixo lista os **tokens** validos da linguagem Java-- reconhecidos pelo Scanner. Os nomes dos terminais sao identicos no **Scanner.flex** e no **Parser.cup** (exigencia do canal de comunicacao **sym**).

Categoria	Lexema(s)	Token (sym)	Atributo
Palavra reservada	program / if / else / while /	PROGRAM, IF, ELSE, int / float WHILE, INT, FLOAT	-
Aritmeticos	+ - * / %	MAIS, MENOS, MULT, DIV, MOD	-
Atribuicao	=	ATRIB	-
Relacionais	== != > >= < <=	IGUAL, DIFER, MAIOR, MAIORIG, MENOR, MENORIG	-
Delimitadores	() { } [] ; ,	ABREPAR, FECHAPAR, ABRECHAVE, FECHACHAVE, ABRECOL, FECHACOL, PTVIRG, VIRG	-
Numero inteiro	[0-9]+	INTEIRO	Double
Numero real	[0-9]+ "." [0-9]+	REAL	Double
Identificador	[a-zA-Z_]([a-zA-Z_] [0-9])*	ID	String
Ignorados	espacos, /* ... */ , // ...	(descartados)	-

Obs.: INTEIRO e REAL sao devolvidos como **Double** para permitir o calculo das expressoes (o tipo do terminal no JFlex deve ser o mesmo definido no JCup).

3. Gramatica e principais regras

A gramatica foi escrita **sem ambiguidade** para evitar conflitos shift-reduce. As principais regras de producao implementadas no **Parser.cup** sao:

```

programa    ::= PROGRAM ID decls bloco

decls       ::= decls decl | (vazio)
decl        ::= tipo lista_ids PTVIRG
tipo        ::= INT | FLOAT | INT '[' ']' | FLOAT '[' ']'
lista_ids   ::= lista_ids VIRG ID | ID

bloco       ::= '{' lista_cmd '}'
lista_cmd   ::= lista_cmd comando | (vazio)
comando     ::= atribuicao | cmd_if | cmd_while | bloco | error PTVIRG

atribuicao   ::= designator ATRIB expr PTVIRG
cmd_if      ::= IF '(' condicao ')' comando [%prec LOWER_THAN_ELSE]
              | IF '(' condicao ')' comando ELSE comando
cmd_while   ::= WHILE '(' condicao ')' comando
condicao     ::= expr (== | != | > | >= | < | <=) expr

expr        ::= expr MAIS expr | expr MENOS expr
              | expr MULT expr | expr DIV expr | expr MOD expr
              | MENOS expr [%prec UMINUS]
              | '(' expr ')' | INTEIRO | REAL | designator
designator   ::= ID | ID '[' expr ']'

```

3.1 Como a ambiguidade foi evitada (precedencia)

Em vez de reescrever a gramatica em varios niveis, usamos as **declaracoes de precedencia** do JCup, que resolvem os conflitos shift-reduce de forma limpa:

```

precedence nonassoc LOWER_THAN_ELSE; // if sem else (dangling-else)
precedence nonassoc ELSE;
precedence left    MAIS, MENOS;      // + e - (menor precedencia)
precedence left    MULT, DIV, MOD;    // *, / e % (maior precedencia)
precedence right   UMINUS;            // menos unario (-5)

```

- **Aritmetica:** * / % tem precedencia maior que + -, e a associatividade a esquerda garante a leitura natural (ex.: $a + b * 2 = a + (b * 2)$).
- **Dangling-else:** o else sempre se liga ao if mais proximo, pois o token ELSE tem precedencia maior que a regra do if-sem-else (LOWER_THAN_ELSE), o que faz o parser dar *shift* no ELSE.

Resultado da geracao do JCup: **0 errors, 0 warnings, 0 conflicts detected** - comprovando que a gramatica esta correta e nao ambigua.

3.2 Acoes semanticas e tratamento de erros

- **Calculo de expressoes:** os terminais/nao-terminais sao tipados (Double) e o resultado e calculado em RESULT. Uma **tabela de simbolos** (HashMap<String, Double>) guarda o valor das variaveis, permitindo expressoes com identificadores (ex.: $x = 5$; $y = x + 3$).
- **Mensagens:** cada regra imprime confirmacoes ("Atribuicao reconhecida", "Comando WHILE reconhecido", "Arquivo sem erros de sintaxe").
- **Recuperacao de erros (modo panico):** a regra comando ::= error PTVIRG faz o parser descartar tokens ate o proximo ;, recuperando-se e continuando a analise.

4. Como compilar e executar

Dentro da pasta **bin**, usando os mesmos comandos da Aula 14:

```
REM 1) Gerar o Parser (parser.java e sym.java)
java -jar java-cup-11b.jar Parser.cup

REM 2) Gerar o Scanner (Scanner.java)
jflex Scanner.flex

REM 3) Compilar tudo
javac -cp ".;java-cup-11b-runtime.jar" *.java

REM 4) Executar sobre o arquivo de teste
java -cp ".;java-cup-11b-runtime.jar" Main entrada.txt
```

Para facilitar, foram incluídos os scripts **compilar.bat** (faz os passos 1 a 3) e **executar.bat** (passo 4).

5. Resultados dos testes

5.1 Teste 1 - Programa valido (entrada.txt)

Exercita declarações de variáveis (int, float e vetor), atribuições, expressões aritméticas com precedência e parênteses, um condicional if-else e um laço while.

Entrada (entrada.txt):

```
program Teste

int a;
int b;
int resultado;
float media;
int[] vetor;

{
    /* atribuições simples e calculo de expressões aritmeticas */
    a = 10;
    b = 20;
    resultado = a + b * 2;
    media = (a + b) / 2;

    /* estrutura condicional if-else */
    if (resultado > 40) {
        resultado = resultado - 5;
    } else {
        resultado = resultado + 5;
    }

    /* laço de repetição while */
    while (a < b) {
        a = a + 1;
    }

    /* expressão com parênteses, precedência e modulo */
    resultado = (a + b) * 2 - 10 % 3;
}
```

Saída (saida.txt):

```

=====
COMPILADOR Java-- | Analise Sintatica (Parser)
Arquivo de entrada: entrada.txt
=====
Declaracao de variavel reconhecida.
Declaracao de variavel reconhecida.
Declaracao de variavel reconhecida.
Declaracao de variavel reconhecida.
Declaracao de variavel reconhecida.
Atribuicao reconhecida: a = 10.0
Atribuicao reconhecida: b = 20.0
Atribuicao reconhecida: resultado = 50.0
Atribuicao reconhecida: media = 15.0
Atribuicao reconhecida: resultado = 45.0
Atribuicao reconhecida: resultado = 50.0
Comando IF-ELSE reconhecido (condicao = true).
Atribuicao reconhecida: a = 11.0
Comando WHILE reconhecido (condicao = true).
Atribuicao reconhecida: resultado = 61.0
=====
Programa 'Teste' reconhecido com SUCESSO!
Arquivo sem erros de sintaxe.
=====

```

Conferindo o calculo: $a + b*2 = 10 + 40 = 50$; $(a+b)/2 = 15$; $(11+20)*2 - 10\%3 = 62 - 1 = 61$. A precedencia e os parenteses funcionaram corretamente.

5.2 Teste 2 - Recuperacao de erro (entrada_erro.txt)

Contem um erro proposital ($y = 5 \ 5$; - falta o operador). O parser detecta o erro, informa linha/coluna, recupera-se em modo panico e **continua** a analise.

Entrada (entrada_erro.txt):

```

program TesteErro

int x;
int y;

{
    x = 10;
    y = 5 5;          /* ERRO PROPOSITAL: faltou o operador entre 5 e 5 */
    x = y + 2;        /* o parser se recupera (modo panico) e continua aqui */
}

```

Saida (saida_erro.txt):

```

=====
COMPILADOR Java-- | Analise Sintatica (Parser)
Arquivo de entrada: entrada_erro.txt
=====
Declaracao de variavel reconhecida.
Declaracao de variavel reconhecida.
Atribuicao reconhecida: x = 10.0
>> ERRO DE SINTAXE na linha 8, coluna 11 (proximo a "5.0")
>> erro de sintaxe recuperado (modo panico).
Atribuicao reconhecida: x = 2.0
=====
Programa 'TesteErro' analisado.
Foram encontrados 1 erro(s) de sintaxe (recuperados em modo panico).
=====

```

6. Conclusao

O Parser desenvolvido cumpre todos os requisitos do trabalho: le um arquivo-fonte Java--, valida a sintaxe a partir de uma gramatica **sem ambiguidade** (0 conflitos no JFlex) e executa acoes

semanticas - com destaque para o **calculo de expressoes** aritmeticas (usando precedencia, parenteses, menos unario e uma tabela de simbolos para variaveis) e para o **tratamento de erros** com recuperacao em modo panico.

Como caracteristica especial, a integracao Scanner+Parser segue fielmente o padrao da Aula 14 (diretivas %cup, %line, %column, %public no JFlex; tipagem de terminais e nao-terminais no JCup). Ressalta-se que, conforme a Aula 15, esta e uma **Traducao Dirigida pela Sintaxe** - as acoes sao disparadas na ordem das reducoes (de baixo para cima), portanto os comandos de controle (if/while) sao **reconhecidos sintaticamente**, sem execucao real do fluxo (sem repeticao de laco).

7. Referencias bibliograficas

- HAUCK, A. Teoria dos Compiladores - Aula 14: Parser JFlex + JCup. FATECS/CEUB.
- HAUCK, A. Teoria dos Compiladores - Aula 15: JCup com Valores (Traducao Dirigida por Sintaxe). FATECS/CEUB.
- HAUCK, A. Trabalho Pratico: Analisador Lexico - Scanner (gramatica Java--). FATECS/CEUB.
- AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. Compiladores: Principios, Tecnicas e Ferramentas. 2. ed. Pearson, 2008.
- JFlex - The Fast Scanner Generator for Java. Manual da versao 1.9.1. Disponivel em: <https://jflex.de/>
- CUP - LALR Parser Generator for Java (Java CUP 11b). Disponivel em: <http://www2.cs.tum.edu/projects/cup/>